g4play: Geometry, Simulation Flow, and NN PID Model

Project documentation

November 3, 2026

1 Geant4 geometry model

The detector model is implemented in `src/waterCherenkovSim.cpp`. The setup consists of:

- World volume: air box ($3 \times 3 \times 4$ m half-lengths).
- Cylindrical stainless-steel tank: outer radius 1.75 m, half-height 2.0 m.
- Steel wall/cap thickness: 4 mm.
- Inner water volume: radius 1.75 m $-$ 4 mm, half-height 2.0 m $-$ 4 mm.

Water optical properties use tabulated refractive index and absorption length versus photon energy. Air and vacuum are assigned optical refractive index tables for optical transport compatibility.

1.1 Photosensor model

Sensors are hemisphere modules covering barrel and endcaps:

- Outer dome radius 4 cm, glass thickness 1 mm.
- Internal vacuum half-sphere.
- Silicon disk at the base: thickness 5 mm, radius 3.8 cm.
- Placement pitch: approximately 10 cm.
- Sensors are oriented inward; barrel placement leaves 10 cm clearance near endcaps.

The silicon optical surface is configured as an ideal absorber (`REFLECTIVITY=0`, `EFFICIENCY=1`), so each optical photon reaching the silicon is registered.

2 Simulation call structure and momentum scan workflow

2.1 Single executable

The CMake build generates:

`./bin/waterCherenkovSim`

Macro files are stored under `configs/`.

2.2 Momentum list scan (`run_scan.py`)

`run_scan.py` reads `configs/run.mac`, then for each requested momentum:

1. rewrites `/gun/particle`,
2. rewrites `/gun/momentumAmp <P> MeV`,
3. writes a temporary macro file,
4. runs:

   ```
   source <geant4.sh> && exec ./bin/waterCherenkovSim <temp_macro>
   ```

5. stores output ROOT as

   ```
   events_<particleTag>_p<P>MeV_<NeventsTag>[suffixes].root
   ```

Default momentum list:

$$[200, 250, 300, 350, 400, 450, 500, 550, 600, 650, 700, 750, 800] \text{ MeV}/c$$

Geant4 environment script selection:

- host `jirikvita`: `/home/qitek/install/geant4-v11.1.1-install/bin/geant4.sh`
- otherwise: `/home/qitek/work/g4/geant4-v11.1.2-install/bin/geant4.sh`

2.3 Parallel particle scans (`superRun.py`)

`superRun.py` launches multiple `run_scan.py` processes in parallel (default particles: e^+, μ^+, π^+). Each process scans the same momentum list (or user-provided list).

3 Poisson smearing and noise model

Readout controls:

```
/g4play/silicon/smearPoissonHits true|false
/g4play/silicon/addNoise true|false
/g4play/silicon/noiseNu <nu>
```

Per module, if raw detected count is N:

- if smearing enabled: $N \leftarrow \text{Poisson}(N)$,
- if additive noise enabled: $N \leftarrow N + \text{Poisson}(\nu)$, where $\nu =$ `noiseNu`.

Output filename tags:

- `_PoissSmear` if smearing is enabled,
- `_addNoise<nu>` if additive noise is enabled.

4 NN architecture and hyperparameters

The NN training pipeline is in `trainNN.py` and feature extraction in `pid_features.py`.

4.1 Input features

For each event, the code aggregates statistics from vector branches in `silicon` and optional `muonDecay` tree:

- count/sum/mean/std/min/max per branch,
- weighted sensor-position means (x, y, z) using `nHits` weights,
- total number of hits,
- indicator whether a `muonDecay` row exists.

Truth class is `primaryPDG`. Momentum target is parsed from filename (`_pXXXMeV_`).

4.2 Model architecture

MLP:

- fully connected hidden layers from `-hidden` (default: `256,128,128`),
- each hidden layer: `Linear -> ReLU -> Dropout`,
- output dimension: $N_{\text{classes}} + 1$,
- outputs = class logits + one normalized momentum regressor.

4.3 Training defaults

- Dropout: 0.1
- Epochs: 100
- Batch size: 64
- Optimizer: Adam
- Learning rate: 10^{-3}
- Momentum loss weight: 1.0
- Seed: 1234
- Data split (stratified): 50% test, remaining 50% split 90/10 into train/val

Loss:

$$\mathcal{L} = \text{CrossEntropy}(\text{PID logits}, y) + \lambda \cdot \text{MSE}(p_{\text{pred,norm}}, p_{\text{true,norm}})$$

with $\lambda =$ `-momentum-loss-weight`.

5 Figures

5.1 Event display snapshots (`EvtDispl/`)

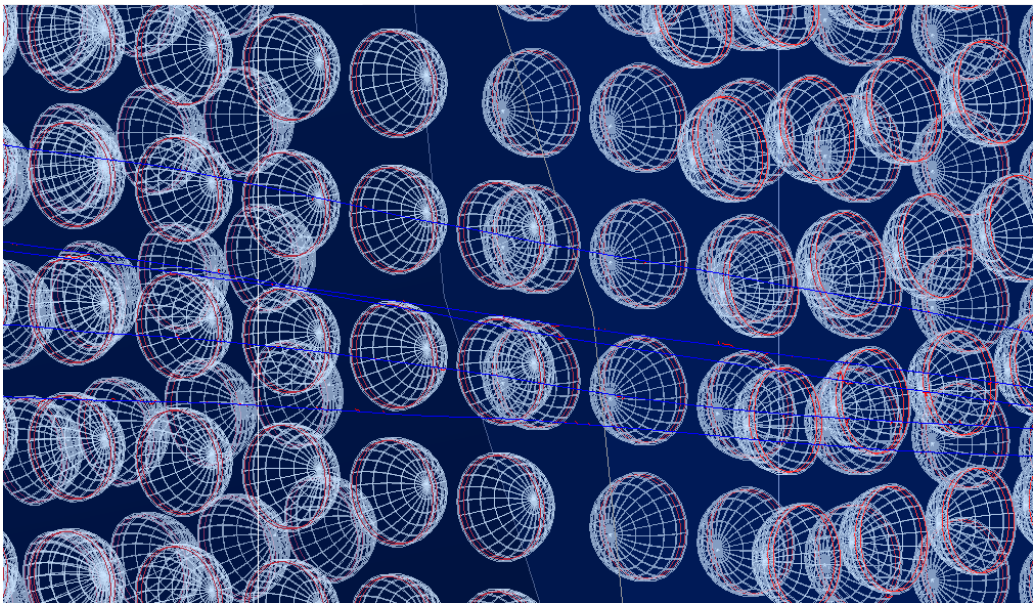


Figure 1: Event display snapshot 1.

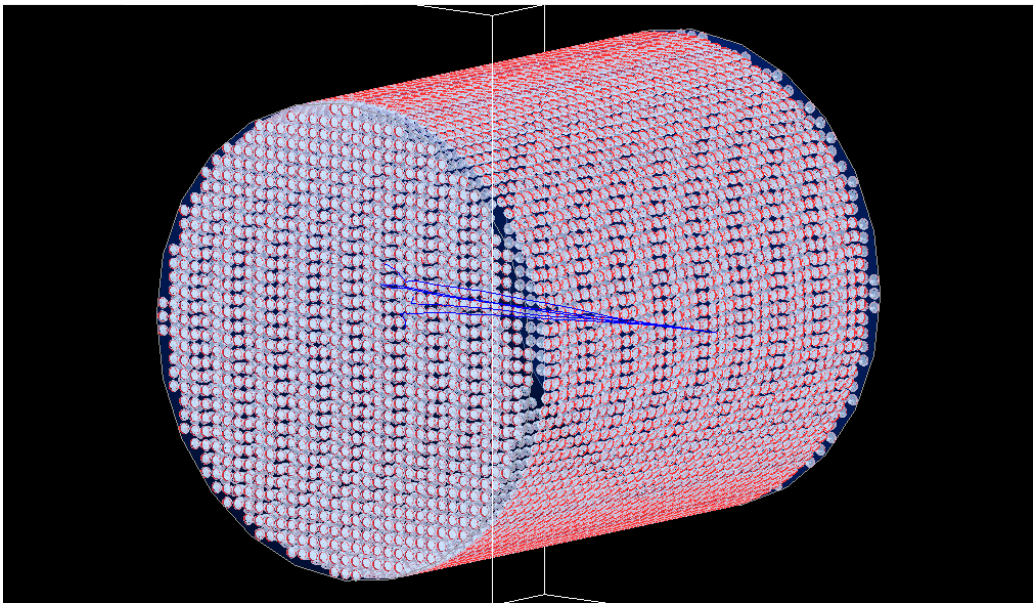


Figure 2: Event display snapshot 2.

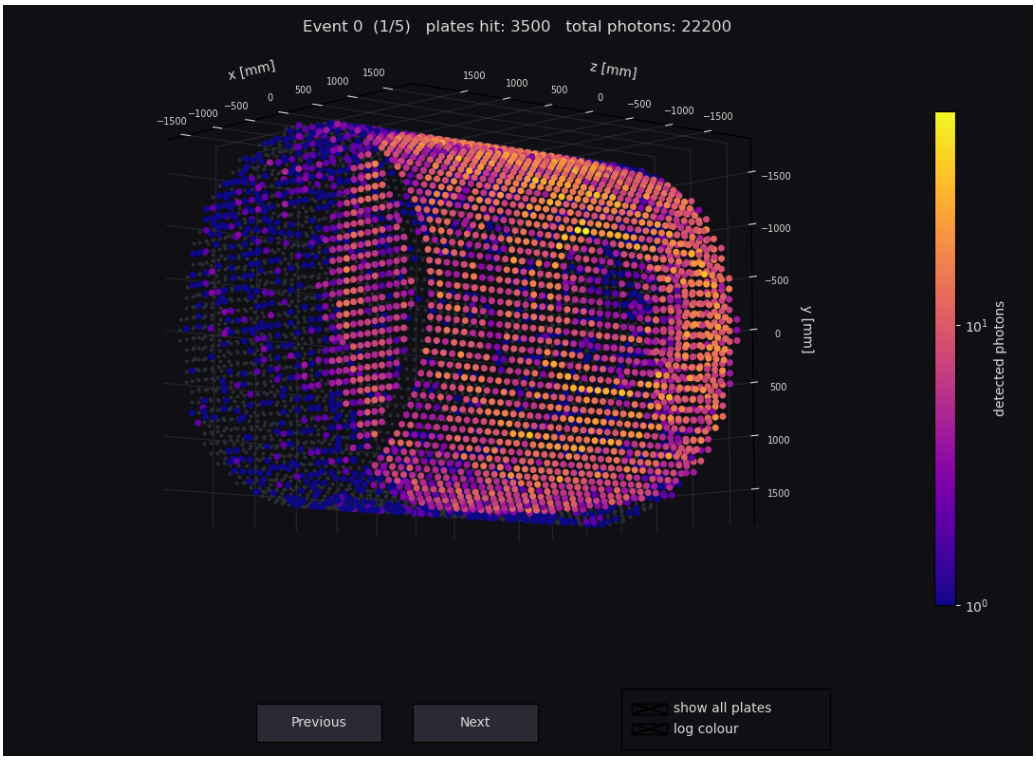


Figure 3: Event display snapshot 3.

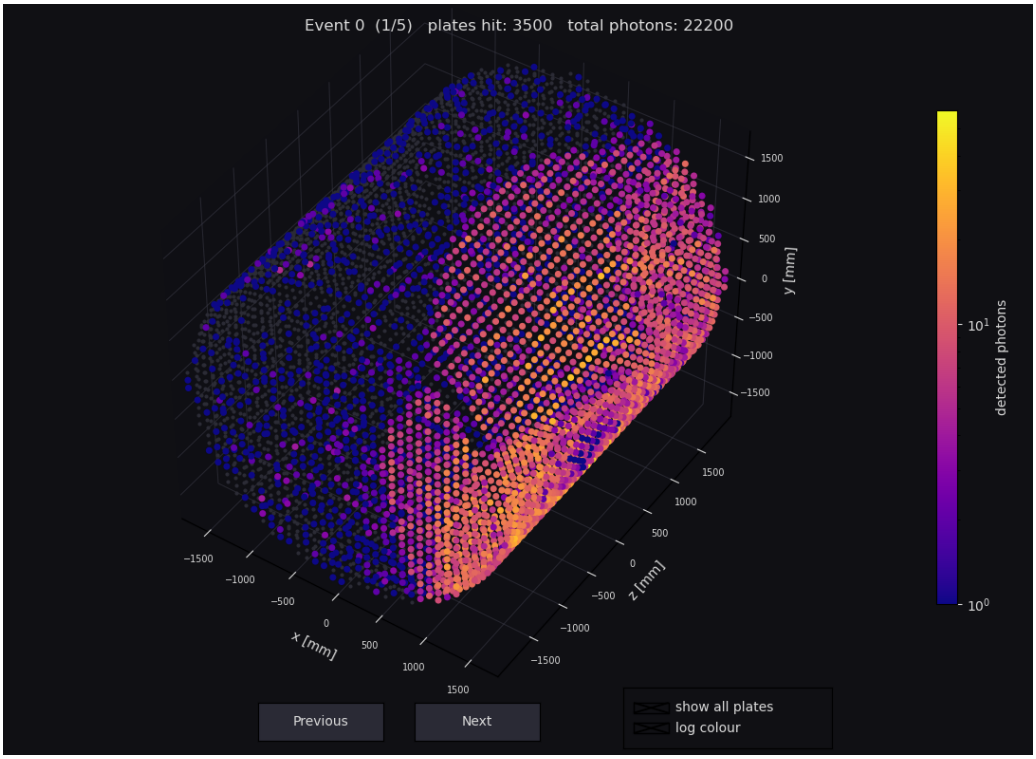


Figure 4: Event display snapshot 4.

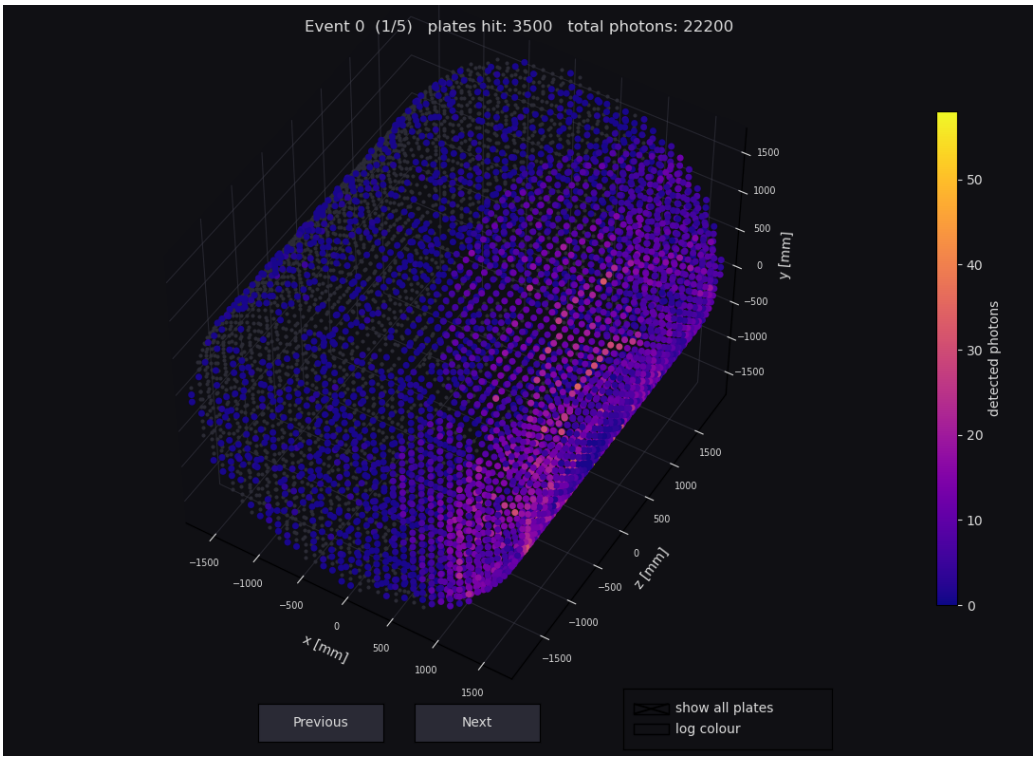


Figure 5: Event display snapshot 5.

5.2 Simulation and analysis screenshots (`Screenshots/`)

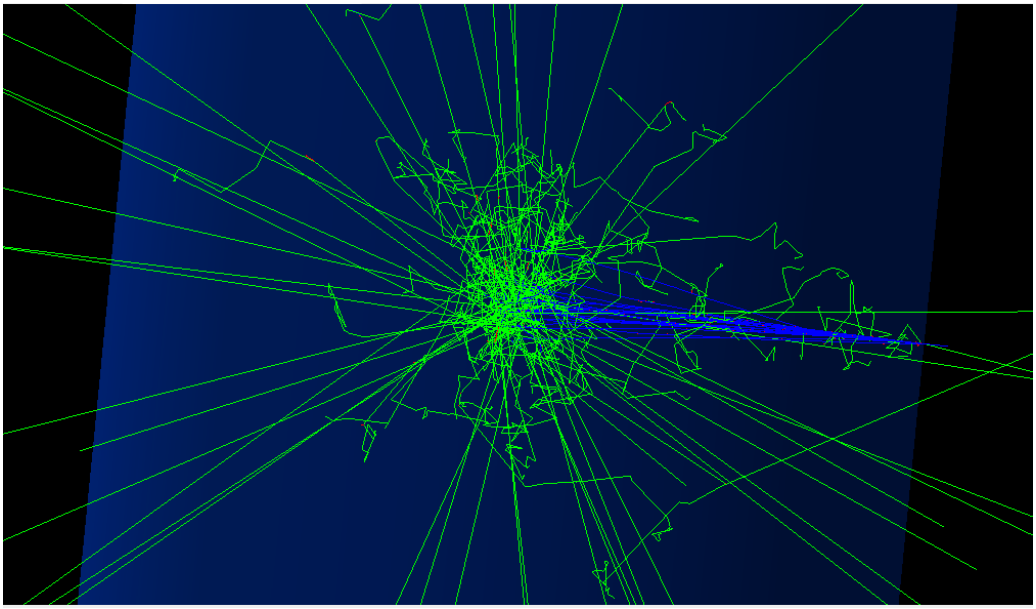


Figure 6: Screenshot 1.

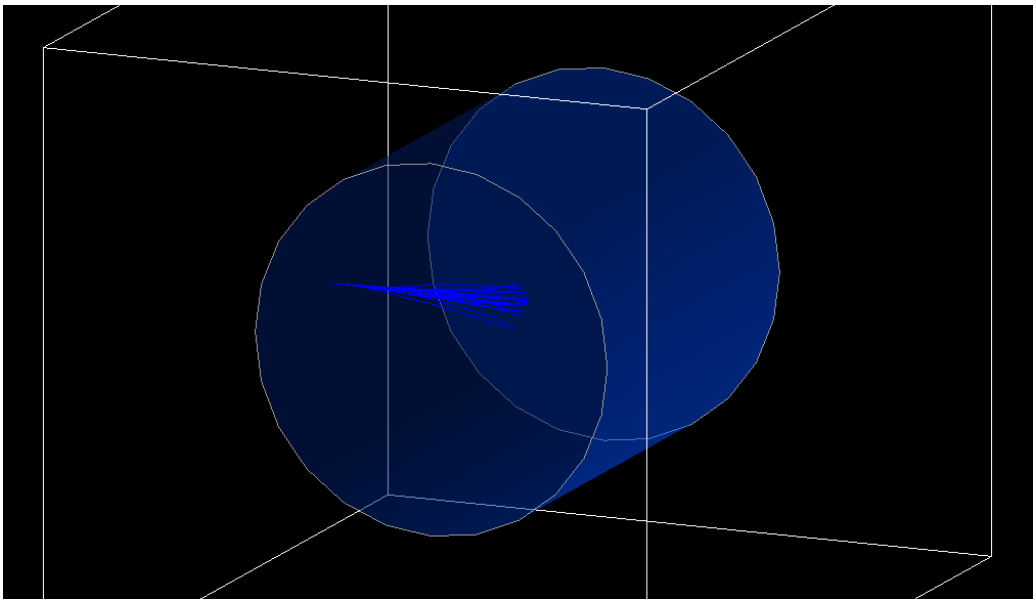


Figure 7: Screenshot 2.

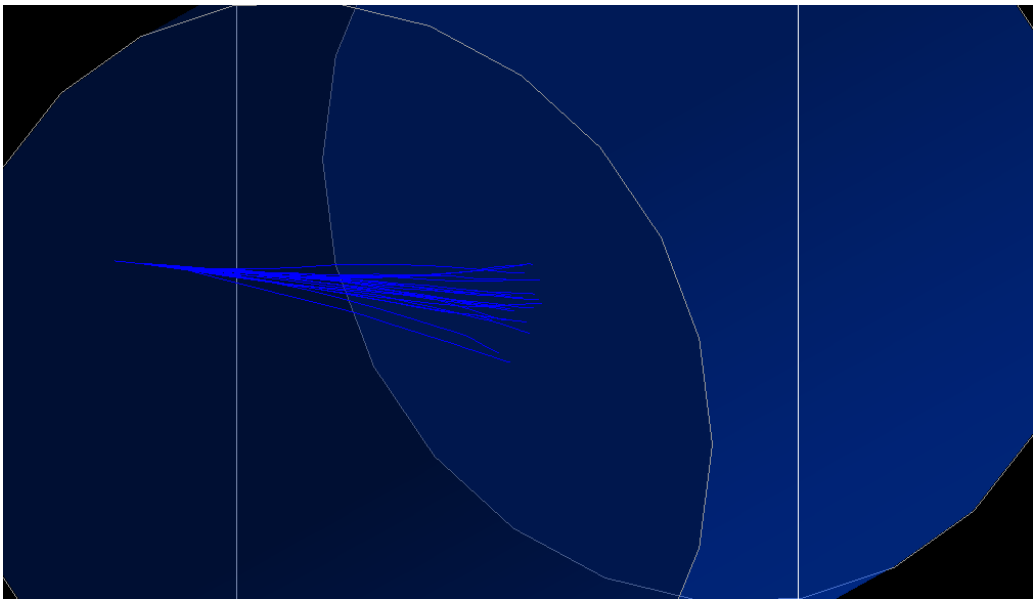


Figure 8: Screenshot 3.

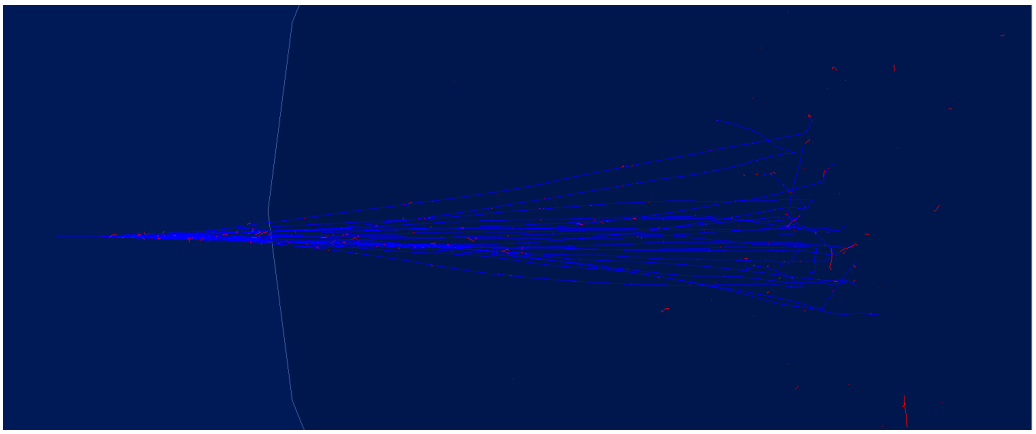


Figure 9: Screenshot 4.

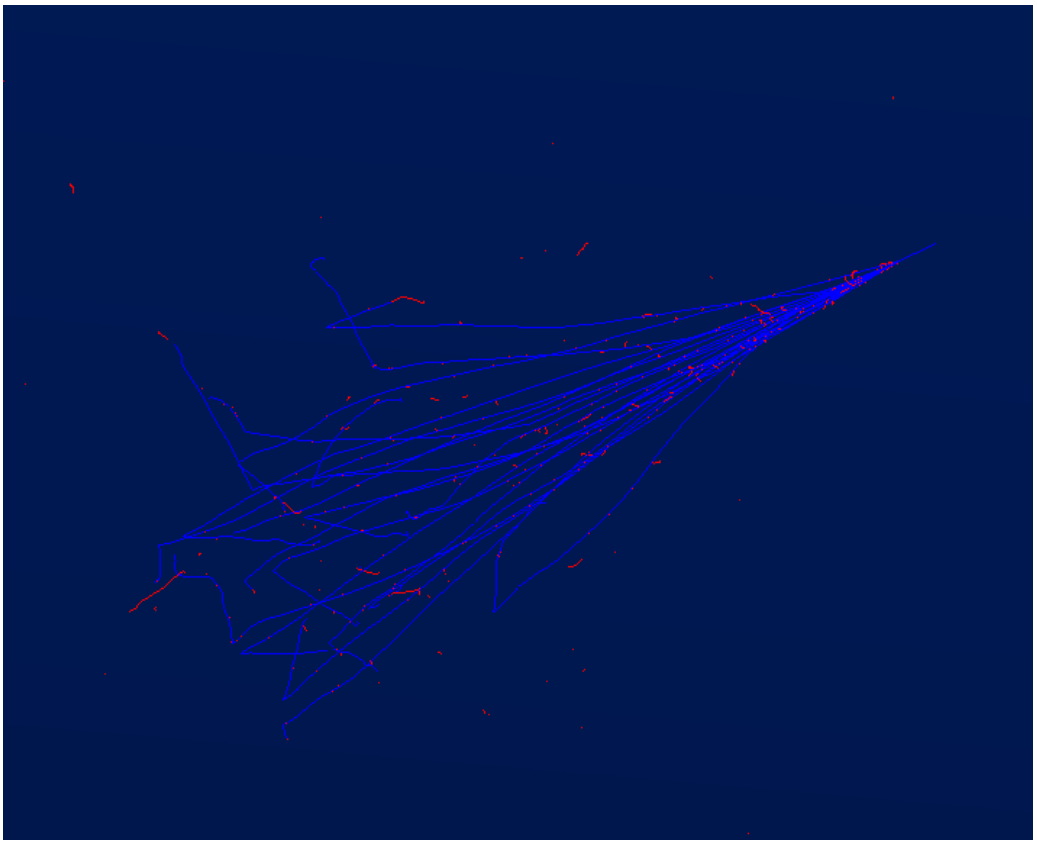


Figure 10: Screenshot 5.

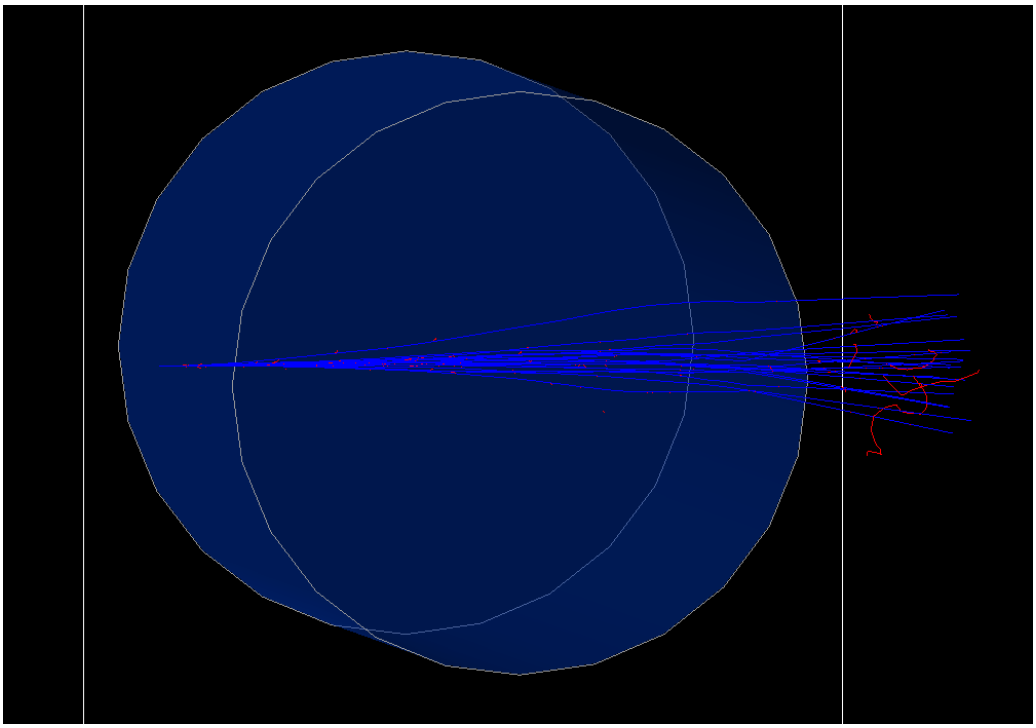


Figure 11: Screenshot 6.

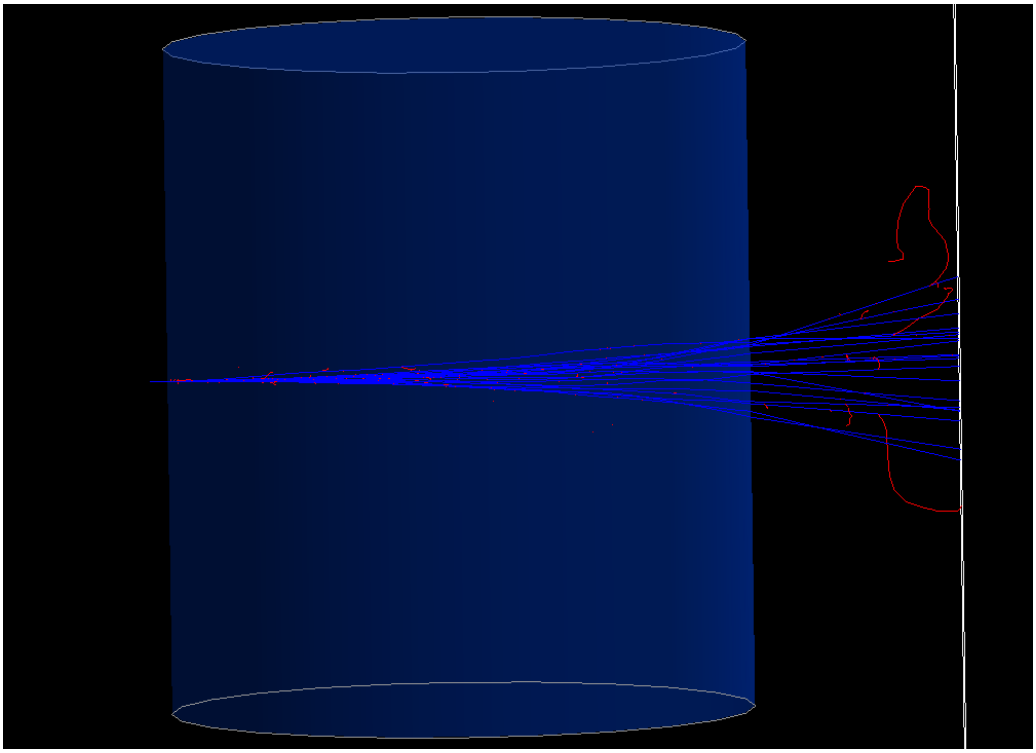


Figure 12: Screenshot 7.

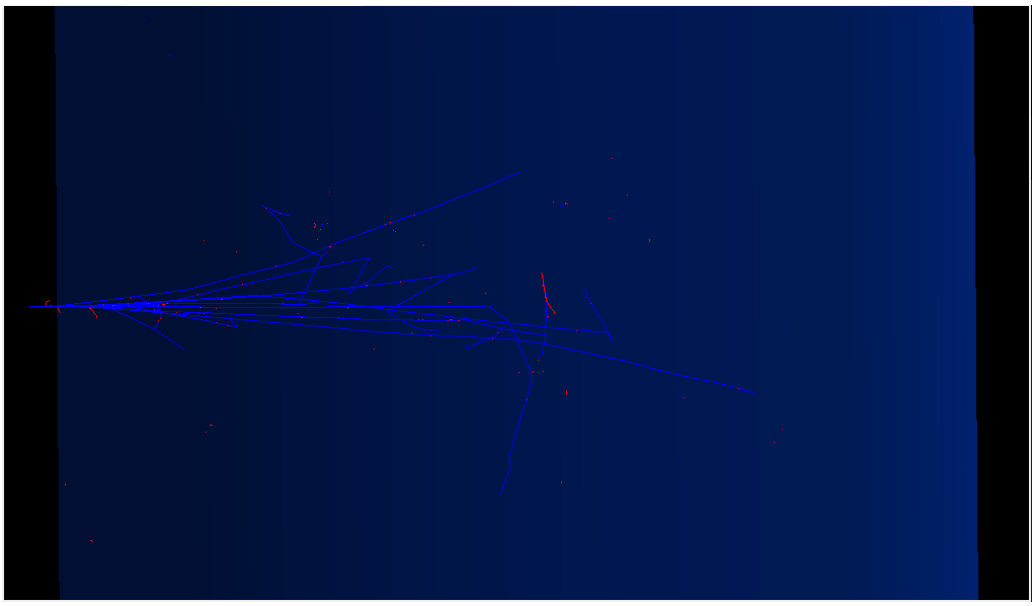


Figure 13: Screenshot 8.

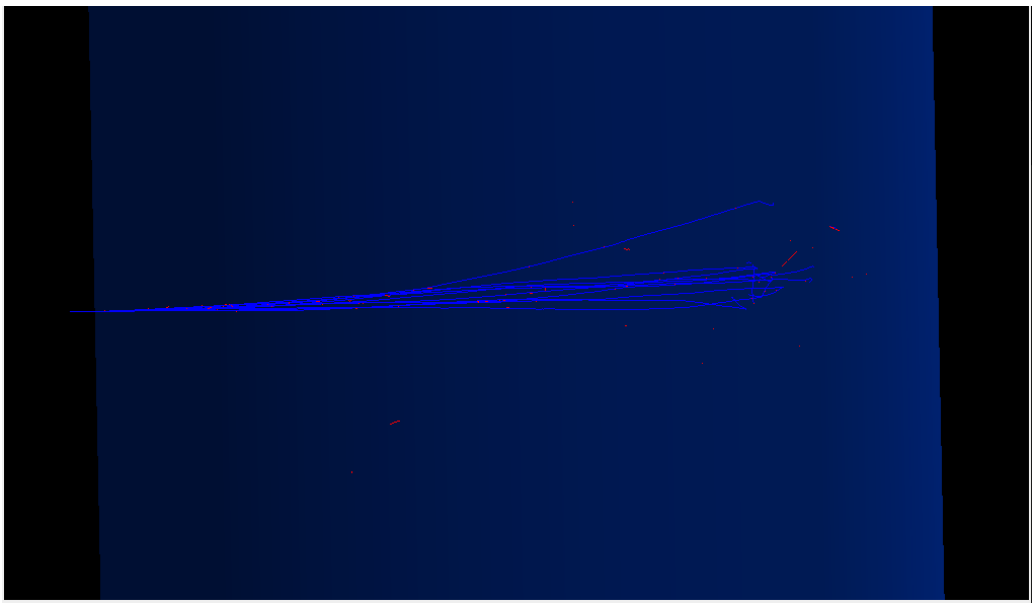


Figure 14: Screenshot 9.

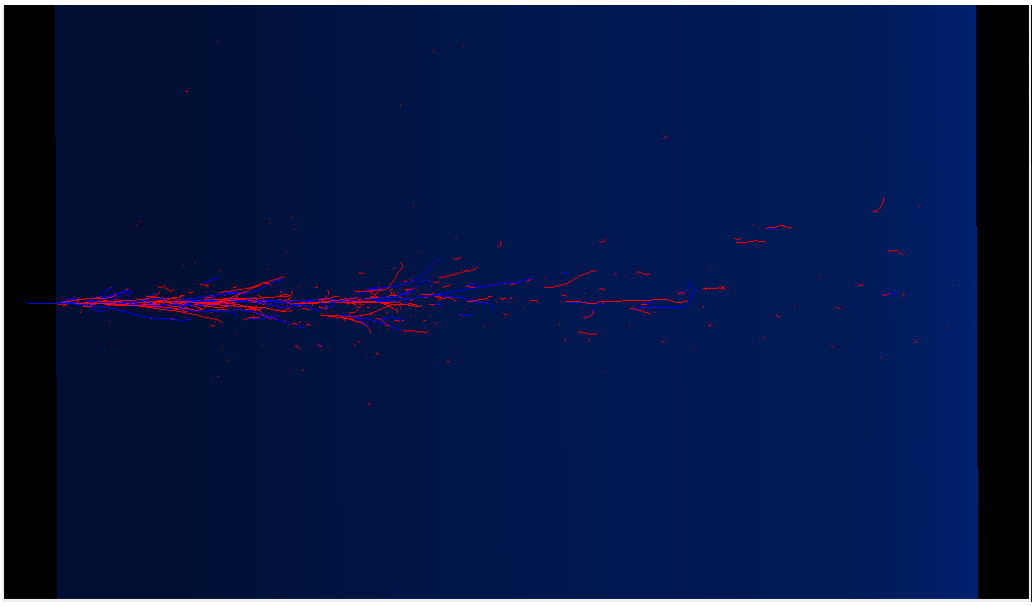


Figure 15: Screenshot 10.